



Republic of the Philippines
BATANGAS STATE UNIVERSITY

The National Engineering University

Alangilan Campus

Golden Country Homes, Alangilan Batangas City, Batangas, Philippines 4200

Tel Nos.: (+63 43) 425-0139 local 2121 / 2221

E-mail Address: coe_alangilan@batstate-u.edu.ph | Website Address: <http://www.batstate-u.edu.ph>

College of Engineering - Department of Electrical Engineering

Bachelor of Science in Computer Engineering

CpE 421: Emerging Technology
2nd Semester A.Y. 2025 – 2026

Group

IoT-Based Prepaid Electricity Management System with Consumption Tracking and Automated Power Cut-off

Instructor:

Engr. Jennifer L. Marasigan

Members:

Achico, Whyner Bonren C.

Cornejo, Carlmartin I.

Frago, Andre Aaron D.

Mulingbayan, Denver Marc T.

Panaligan, Justin A.

MAY 2026

Executive Summary

The Dynamic Prepaid Electricity Management System is an advanced IoT framework designed to decentralize and modernize multi-tenant residential energy sub-metering. Utilizing a dual-microcontroller architecture, the system employs an Arduino Uno as the primary logic engine and an ESP-12E NodeMCU configured as a transparent serial bridge, ensuring seamless, low-latency communication with the Blynk cloud platform. By simulating dynamic electrical loads across a dual-house microgrid using independently controlled LED arrays, the project successfully implements a real-time, software-defined credit algorithm. This logic continuously calculates proportional power draw and forecasts temporal balance exhaustion, triggering an automated hardware kill-switch and asynchronous dashboard synchronization the moment a user's prepaid credits are depleted. Unlike conventional metrology solutions, this system leverages a highly granular mobile interface, empowering users with immediate transparency over individual appliance consumption, active power loads, and real-time financial tracking. Testing successfully demonstrated that this streamlined, hardware-efficient architecture effectively prevents unexpected utility costs and scales reliably to manage multi-node environments, offering a highly responsive prototype for localized smart-grid deployment.

Introduction

In the modern household, electricity has become an indispensable resource that powers everything from essential appliances to our digital lives. However, the way we manage and pay for this utility has largely remained stagnant. For most of us, electricity is a "blind" expense we consume it freely throughout the month and only face the reality of our usage once the monthly bill arrives. As engineering students, we observed that this lack of real-time awareness often leads to financial stress and wasteful consumption habits, as there is no immediate feedback to tell us how much each appliance is actually costing us at the moment.

The group decided to develop the Prepaid Electricity Management System to bridge the gap between traditional utility services and the digital age. We envisioned a system that operates with the same ease and transparency as a prepaid mobile phone service, where you only use what you have already budgeted for. By leveraging Internet of Things technology, we aimed to create a solution that gives consumers a voice in their energy management. We wanted to transform the consumer from a passive bill-payer into an active, informed user who can monitor their consumption at the touch of a button through their personal devices, ensuring that every watt used is intentional and accounted for.

Problem Statement

The current post-paid electricity billing system in our local community presents several significant challenges that affect both households and property owners. The most prominent issue is "bill shock," where households are surprised by unexpectedly high charges at the end of the month because they had no means of tracking their consumption in real-time. Without a digital interface, consumers are forced to manually interpret complicated analog meters, a task that is both inconvenient and prone to error.

Furthermore, traditional meters offer no way to set limits or receive warnings before a certain budget is exceeded. This lack of control is especially problematic for landlords of rental properties and dormitories, who often struggle to collect utility payments or manage shared expenses fairly. Currently, there is no accessible, low-cost device that allows a user to remotely shut off their power or receive an instant alert when their budget is running low. These gaps in the existing infrastructure inspired us to build a prototype that combines automation, remote monitoring, and automated credit deduction into a single, cohesive system that addresses the lack of transparency in the energy sector without requiring complex on-site display hardware.

Objective and Scope

- **Primary Objective:** To develop a standalone wireless electricity management system that integrates IoT technology for automated and manual power control based on monetary credit.

Leading Innovations, Transforming Lives, Building the Nation

- **Specific Objective 1:** To implement real-time tracking of power consumption
- **Specific Objective 2:** To design an automated cut-off mechanism that triggers when the virtual balance reaches zero.
- **Specific Objective 3:** To provide a user-friendly mobile interface for reloading units and monitoring usage through the Blynk application.
- **Scope:** The project is limited to a breadboard + ESP-12E prototype utilizing 4 LEDS each house to simulate real-world scenario

System Design and Architecture

The architecture is divided into two main sections: the Control Board and the Load Board. This division ensures signal integrity and prevents interference between the microcontrollers and the fluctuating current of the load sensors.

- **Control Unit:** This serves as the "brain" of the project, consisting of an Arduino Uno for local logic processing and an ESP-12E NodeMCU for WiFi communication. The two microcontrollers communicate via a software serial bridge (TX/RX), allowing the Arduino to pass raw data to the ESP8266 for cloud uploading
- **Sensing and Actuation:** An Arduino Uno is wired via BreadBoard Power Supply that communicates with the Esp module to measure remaining balance. The Arduino Uno also serves as the primary actuator, acting as a master switch that can disconnect the entire circuit based on the software's credit logic.
- **User Interface and Indicators:** Remote monitoring is handled entirely through the Blynk mobile application. Locally, the system uses a 4-LED "House" indicator system to show active loads and a dedicated Remaining Balance to give insights whether the credits are low.

Development Process and Tools Used

The development followed a multi-phase hardware-software integration approach:

- **Firmware Development:** The team used the Arduino IDE to program the microcontrollers in C++, utilizing libraries such as BlynkSimpleEsp8266 for cloud integration .
- **Hardware Assembly:** The system was built on 1 breadboard, using 220Ohms resistors for LED current limiting and jumper wires for internal connections .
- **Cloud Configuration:** 16 datastreams (V1-V16) were established in the Blynk Web Dashboard to track variables like "Credits Remaining" and "Estimated Time Remaining".

Implementation of Emerging Technologies

- **Internet of Things (IoT):** The system functions as an active IoT node, bridging physical electrical loads to a digital mobile dashboard.
- **Embedded Systems:** The firmware implements real-time sensor polling and a \$10000.0\$ deduction scalar to simulate local energy pricing without a persistent internet connection.
- **Smart Home Automation:** By retrofitting standard IoT devices with light circuits, the project demonstrates the feasibility of low-cost home automation.

Testing Procedures and Results

Testing occurred in four phases. First, Wifi Verification confirmed that the ESP can indeed act as a wifi modem for the Arduino. Second, the serial bridge test verified the ESP-12E correctly uploaded data to Blynk. Third, the logic test demonstrated that a virtual balance was correctly deduced over time. When the balance reached zero, the Arduino successfully disconnected the load, and the dashboard displays 0 remaining balance. Fourth, standalone verification confirmed the 9V adapter could power the entire system without a PC.

Analysis and Discussion

The prototype successfully met its functional goals, proving that precise energy tracking can be localized through affordable components. A major focal point of this project was optimizing the processing load. By utilizing the ESP-12E strictly as a "Transparent Serial Bridge" running pure Wi-Fi connection logic, we completely eliminated the overhead of parsing standard Espressif AT commands.

This architecture allowed the Arduino Uno to act as a dedicated central processing unit (CPU) for metrology and state logic. Passing raw serial bytes via BlynkSimpleStream.h proved to be highly stable over SoftwareSerial at 9600 baud. It ensured that network latency or Wi-Fi reconnect attempts on the ESP-12E did not block the Arduino's critical core task: calculating the live power draw every 2 seconds.

Conclusion

The system successfully demonstrates that integrating IoT frameworks, wireless communication protocols, and accessible embedded systems can effectively resolve residential budgeting challenges. By utilizing a transparent serial bridge architecture, the project proved that low-cost hardware can reliably execute real-time power monitoring and cloud synchronization without sacrificing stability. Furthermore, the dynamic load algorithm offers a transparent alternative to opaque, proprietary sub-meters by giving users real-time visibility into their projected consumption. Ultimately, by seamlessly integrating automated budget enforcement (auto-kill switches) with manual smartphone control, this prototype delivers a highly scalable and balanced approach to modernizing multi-tenant utility management.

Recommendations

For future improvements, we recommend the following:

- **LCD Integration:** Adding a 16x2 LCD in future iterations to provide local diagnostics for users without immediate smartphone access.
- **Payment API Integration:** Integrating a mobile payment API (such as GCash or Maya) to allow users to purchase electricity credits directly within the app.
- **Data Redundancy:** Implementing an SD card module for local data logging to ensure that energy consumption data is preserved even if the Wi-Fi connection is lost.
- **Predictive AI:** Exploring the use of machine learning to analyze usage patterns and predict "time-to-empty" based on historical habits.

References:

Chaudhary, S., Kumar, A., & Kaundal, V. (2026). *GSM-Based Prepaid Meter*. Project Report on Real-Time Energy Monitoring Systems. [Link](#)

Hseiki, H., El-Hajj, A. M., Ajra, Y., Hija, F. A., & Haidar, A. M. (2023). *A Secure and Resilient Smart Energy Meter*. IEEE Access. [Link](#)

Miller, T., Mikiciuk, G., et al. (2025). *The IoT and AI in Agriculture: A Systematic Review of Smart Sensing Technologies*. Sensors, 25(12), 3583. [Link](#)

Rizkyanto, et al. (2024). *Calibration and Validation of INA219 as Sensor Power Monitoring System using Linear Regression*. ResearchGate. [Link](#)

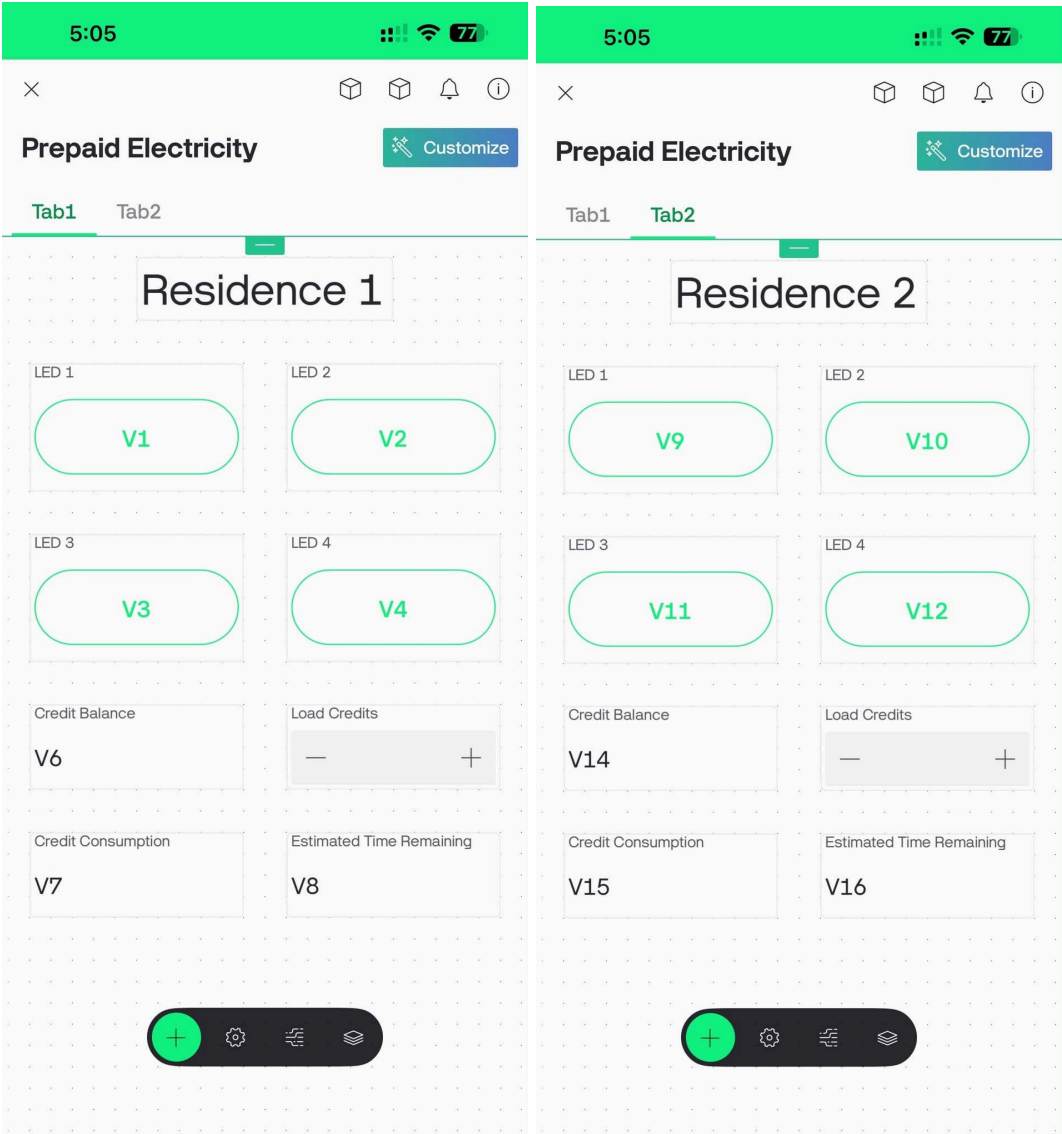
Umamaheswari, S., et al. (2025). *Smart Prepaid Electric Meter System using IoT*. IEEE International Conference on Electrical, Computer and IoT Technologies. [Link](#)

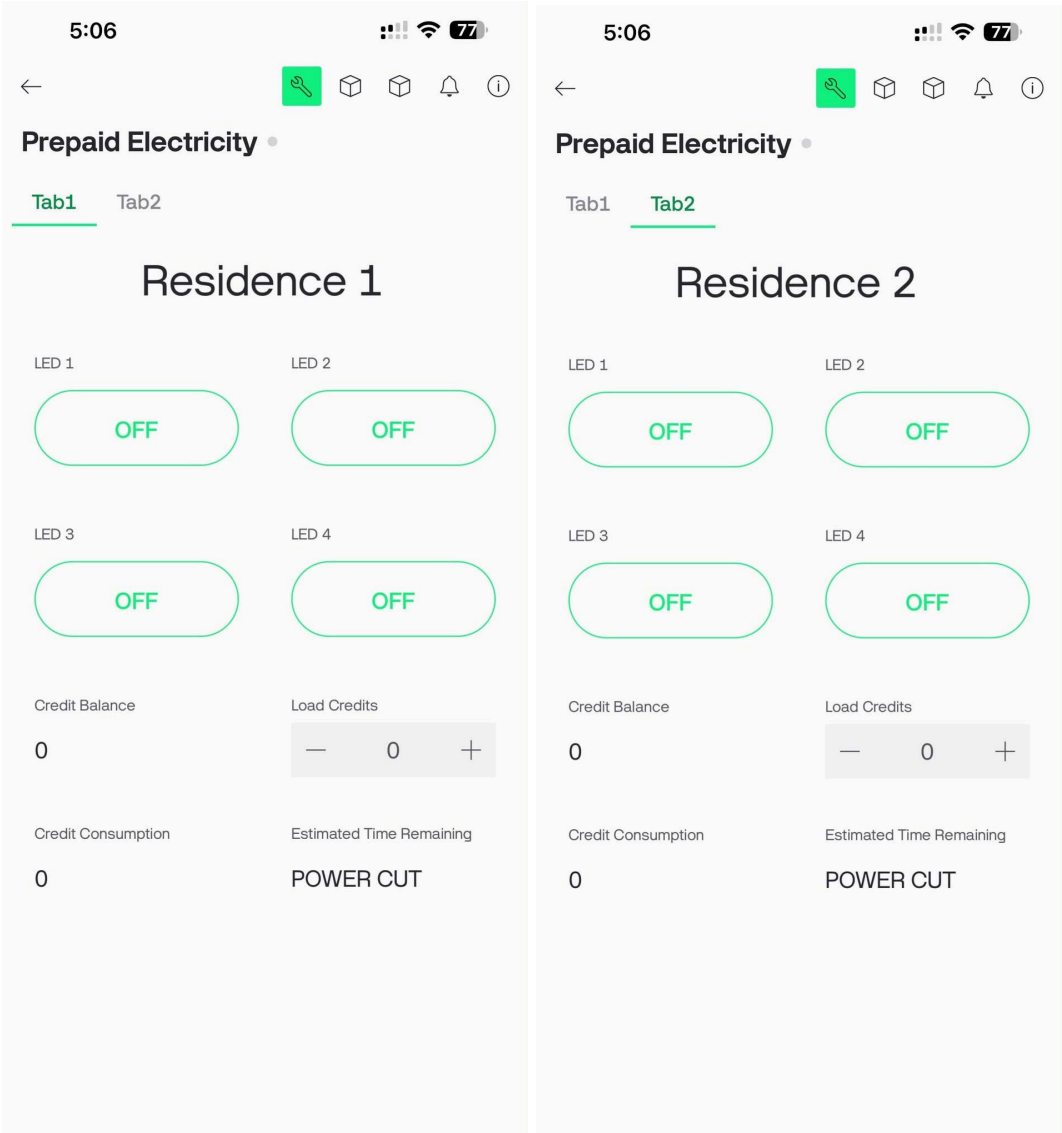
Yan, L., & Wen, J. (2021). *Current-differential techniques as a reliable indicator of meter bypass*. RSIS International. [Link](#)

Appendicies

Appendix A: Datastream Mapping Table

- Virtual Pin Setup:





Appendix B: Code

Arduino Code:

```
/*
 * MAIN BRAIN: Arduino Uno (Dynamic Load Smart Grid)
 * Architecture: Blynk Stream over SoftwareSerial via ESP-12E Bridge
 */

#define BLYNK_TEMPLATE_ID "TMPL68w52Ri5o"
#define BLYNK_TEMPLATE_NAME "Prepaid Electriccity"
#define BLYNK_AUTH_TOKEN "9Kgys92P1Gbfql9JqFUapDK8CH-QFX4c"
#define BLYNK_PRINT Serial

#include <SoftwareSerial.h>
#include <BlynkSimpleStream.h>

SoftwareSerial EspSerial(2, 3); // RX, TX for ESP-12E Bridge
```

```
// --- Hardware Pins ---

const int pinsA[4] = {4, 5, 6, 7};    // House A LEDs
const int pinsB[4] = {8, 9, 10, 11};  // House B LEDs


// --- House States ---

bool stateA[4] = {false, false, false, false}; // Tracks intended state of House
A LEDs

float balanceA = 0.0;

float drawA = 0.0;


bool stateB[4] = {false, false, false, false}; // Tracks intended state of House
B LEDs

float balanceB = 0.0;

float drawB = 0.0;


// Power consumption per active LED per 2-second cycle

const float COST_PER_LED = 0.2;


BlynkTimer timer;


void setup() {

    Serial.begin(9600);

    EspSerial.begin(9600);

    // Initialize all LED pins to OFF

    for (int i = 0; i < 4; i++) {

        pinMode(pinsA[i], OUTPUT);

        digitalWrite(pinsA[i], LOW);

        pinMode(pinsB[i], OUTPUT);

        digitalWrite(pinsB[i], LOW);

    }

    Serial.println("--- Dynamic Load Meter Started ---");

    delay(5000);

    Blynk.begin(EspSerial, BLYNK_AUTH_TOKEN);
```

Leading Innovations, Transforming Lives, Building the Nation

```

    // Run the core metering cycle every 2 seconds
    timer.setInterval(2000L, manageSmartGrid);
}

void loop() {
    Blynk.run();
    timer.run();
}

// =====
// BLYNK INPUT HANDLERS
// =====

// House A: LED Switches
BLYNK_WRITE(V1) { stateA[0] = param.asInt(); }
BLYNK_WRITE(V2) { stateA[1] = param.asInt(); }
BLYNK_WRITE(V3) { stateA[2] = param.asInt(); }
BLYNK_WRITE(V4) { stateA[3] = param.asInt(); }

// House A: Top-Up
BLYNK_WRITE(V5) {
    float topUp = param.asFloat();
    if (topUp > 0) {
        balanceA += topUp;
        Blynk.virtualWrite(V6, balanceA);
    }
}

// House B: LED Switches
BLYNK_WRITE(V9) { stateB[0] = param.asInt(); }
BLYNK_WRITE(V10) { stateB[1] = param.asInt(); }
BLYNK_WRITE(V11) { stateB[2] = param.asInt(); }
BLYNK_WRITE(V12) { stateB[3] = param.asInt(); }

// House B: Top-Up

```



```

BLYNK_WRITE(V13) {

    float topUp = param.asFloat();

    if (topUp > 0) {

        balanceB += topUp;

        Blynk.virtualWrite(V14, balanceB);

    }

}

// =====
// CORE METERING LOGIC
// =====

void manageSmartGrid() {

    // 1. CALCULATE DYNAMIC POWER DRAW

    int activeLedsA = stateA[0] + stateA[1] + stateA[2] + stateA[3];

    int activeLedsB = stateB[0] + stateB[1] + stateB[2] + stateB[3];

    drawA = activeLedsA * COST_PER_LED;

    drawB = activeLedsB * COST_PER_LED;

    // 2. PROCESS HOUSE A

    if (balanceA > 0) {

        balanceA -= drawA;

        if (balanceA <= 0) {

            balanceA = 0; // Prevent negative balance

            killPower('A'); // Cut power if balance depleted mid-cycle

        } else {

            // Balance is good, apply intended LED states

            for(int i=0; i<4; i++) digitalWrite(pinsA[i], stateA[i] ? HIGH : LOW);

        }

    } else if (activeLedsA > 0) {

        // Balance is 0 but user tried to turn a switch on

        killPower('A');

    }

    // 3. PROCESS HOUSE B

```

```

if (balanceB > 0) {

    balanceB -= drawB;

    if (balanceB <= 0) {

        balanceB = 0;

        killPower('B');

    } else {

        for(int i=0; i<4; i++) digitalWrite(pinsB[i], stateB[i] ? HIGH : LOW);

    }

} else if (activeLedsB > 0) {

    killPower('B');

}

// 4. CALCULATE ESTIMATED TIME REMAINING

String timeA = calculateTimeLeft(balanceA, drawA);
String timeB = calculateTimeLeft(balanceB, drawB);

// 5. UPDATE BLYNK DASHBOARD

Blynk.virtualWrite(V6, balanceA);
Blynk.virtualWrite(V7, drawA);
Blynk.virtualWrite(V8, timeA);

Blynk.virtualWrite(V14, balanceB);
Blynk.virtualWrite(V15, drawB);
Blynk.virtualWrite(V16, timeB);
}

// =====
// HELPER FUNCTIONS
// =====

// Calculates remaining time and formats as MM:SS
String calculateTimeLeft(float balance, float draw) {

    if (balance <= 0) return "POWER CUT";

    if (draw == 0) return "Standby (No Draw)";

    // Total cycles left * 2 seconds per cycle

```

```

int totalSeconds = (balance / draw) * 2;

int mins = totalSeconds / 60;

int secs = totalSeconds % 60;

return String(mins) + "m " + String(secs) + "s";
}

// Forcefully shuts off LEDs and resets App switches
void killPower(char house) {

  if (house == 'A') {

    for(int i=0; i<4; i++) {

      digitalWrite(pinsA[i], LOW); // Turn off physical LED

      stateA[i] = false;           // Reset internal logic state

      Blynk.virtualWrite(V1 + i, 0); // Flip Blynk switch to OFF visually

    }

  } else if (house == 'B') {

    for(int i=0; i<4; i++) {

      digitalWrite(pinsB[i], LOW);

      stateB[i] = false;

      Blynk.virtualWrite(V9 + i, 0);

    }

  }

}
}

```

ESP-12E Code:

```
// MODEM MODE: Transparent Bridge
```

```

#include <ESP8266WiFi.h>

char ssid[] = "Kuroshi";

char pass[] = "kasiiloveyou"; // Update this!

const char* server = "blynk.cloud";

const int port = 80;

WiFiClient client;

void setup() {

  Serial.begin(9600); // 9600 baud for Arduino compatibility

```

Leading Innovations, Transforming Lives, Building the Nation

```
WiFi.disconnect();

// WiFi.setOutputPower(5); // Keep this commented out if it helped connection!

delay(100);

WiFi.mode(WIFI_STA);

WiFi.begin(ssid, pass);

while (WiFi.status() != WL_CONNECTED) {

    delay(500);

}

}

void loop() {

    if (!client.connected()) {

        if (!client.connect(server, port)) {

            delay(500);

            return;

        }

    }

    if (Serial.available()) {

        client.write(Serial.read());

    }

    if (client.available()) {

        Serial.write(client.read());

    }

}
```